Ex4a: Information Retrieval using NLTK

1. Learning Objectives

- To understand the basic concept of Information Retrieval (IR).
- To preprocess text using NLTK.
- To perform lowercasing, punctuation removal, tokenization, and stop-word removal.
- To convert text documents into numerical vectors using TF-IDF.
- To calculate cosine similarity between a user query and reviews.
- To retrieve the top-k most relevant reviews.
- To understand how text preprocessing improves information retrieval.

2. Methods Used / Python Libraries Involved

Library / Method	Purpose	Reason to Use
Pandas	Load and manipulate the dataset	Easy handling of CSV data and DataFrames
NLTK	Natural Language Processing	Provides tokenization and stop-word removal
word_tokenize()	Splits text into individual words	Required for text preprocessing
stopwords	Provides common English stopwords	Provides common English stopwords
TfidfVectorizer	Converts text into TF-IDF vectors	Represents text numerically based on word importance

3. Program Code

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

df = pd.read_csv("Reviews.csv")
df = df[['Text']]
df = df.dropna()
df = df.head(10000)

print("Number of reviews:", len(df))
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    words = word_tokenize(text)
    words = [word for word in words if word not in stop_words]
    return ' '.join(words)

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['Cleaned_Text'])
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)

def retrieve_reviews(query, top_k=5):
    cleaned_query = preprocess_text(query)
    query_vector = vectorizer.transform([cleaned_query])
    similarity_scores = cosine_similarity(
        query_vector,
        Tfidf_matrix
```

```
).flatten()
    top_indices = similarity_scores.argsort()[-top_k:][::-1]
    print("\nQuery:", query)
    print("\nTop", top_k, "Relevant Reviews:\n")

    for i in top_indices:
        print("Similarity Score:",
              round(similarity_scores[i], 4))
        print("Original Review:")
        print(df.iloc[i]['Text'])
        print("\nCleaned Review:")
        print(df.iloc[i]['Cleaned_Text'])
        print("-" * 80)

retrieve_reviews("great product with fast shipping", 5)
retrieve_reviews("disappointed", 5)
```

4. Output

```
Number of reviews: 20
TF-IDF Matrix Shape: (20, 44)

Query: great product with fast shipping

Top 5 Relevant Reviews:

Similarity Score: 0.8538
Original Review:
This is a great product and it arrived with fast shipping.

Cleaned Review:
great product arrived fast shipping
--------------------------------------------------------------------------------
------
Similarity Score: 0.5296
Original Review:
The shipping was fast and the product was exactly as described.

Cleaned Review:
shipping fast product exactly described
--------------------------------------------------------------------------------
------
Similarity Score: 0.5055
```

```
Original Review:
Great product, excellent quality and very fast delivery.

Cleaned Review:
great product excellent quality fast delivery
--------------------------------------------------------------------------
------
Similarity Score: 0.415
Original Review:
Fast shipping and good packaging. I am happy with the purchase.

Cleaned Review:
fast shipping good packaging happy purchase
--------------------------------------------------------------------------
------
Similarity Score: 0.3138
Original Review:
The delivery was fast but the product was average.
```

5. Learning Outcome

- I learned how to build a basic Information Retrieval system using Python.
- Learned how to preprocess review text using NLTK.
- Understood the importance of tokenization and stop-word removal.
- Learned how TF-IDF represents text as numerical vectors.
- Learned how cosine similarity measures the similarity between a query and documents.
- Successfully retrieved the top-k relevant Amazon product reviews.
- Understood how similarity scores can be used to rank search results.
- Gained practical knowledge of applying NLP techniques to real-world text data.

Ex4b: Information Retrieval using spaCy

1. Learning Objectives

- To understand Information Retrieval (IR) using the spaCy NLP library.
- To preprocess Amazon Fine Food Reviews using tokenization, lowercasing, stopword removal, non-alphabetic filtering, and lemmatization.
- To convert cleaned reviews into TF-IDF vectors.
- To preprocess a user query using the same NLP pipeline.
- To calculate cosine similarity between the query and reviews.
- To retrieve the top-k most relevant reviews.
- To analyze the retrieved reviews along with their similarity scores and metadata.

2. Methods Used / Python Libraries Involved

Library / Method	Purpose	Reason to Use
Pandas	Load and manipulate the dataset	Makes it easy to handle CSV data and DataFrames
spaCy	Perform NLP preprocessing	Provides efficient tokenization, stopword detection, and lemmatization
token.is_alpha	Filter tokens	Removes numbers, punctuation, and special characters
token.lemma_	Perform lemmatization	Converts words into their base/root form
argsort()	Sort similarity scores	Helps identify the top-k relevant reviews

3. Program Code

```
import pandas as pd
import spacy

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load("en_core_web_sm")

df = pd.read_csv("Reviews.csv")

df = df.dropna(subset=["Text"])

df = df.head(20)

print("Number of reviews:", len(df))


def preprocess_text(text):

    doc = nlp(text.lower())

    cleaned_words = []

    for token in doc:

        if token.is_alpha and not token.is_stop:
            cleaned_words.append(token.lemma_)

    return " ".join(cleaned_words)


df["Cleaned_Text"] = df["Text"].apply(preprocess_text)


vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])

print("TF-IDF Matrix Shape:", tfidf_matrix.shape)


def retrieve_reviews(query, top_k=5):
```

```
    cleaned_query = preprocess_text(query)

    print("\nOriginal Query:", query)
    print("Processed Query:", cleaned_query)

    query_vector = vectorizer.transform([cleaned_query])

    similarity_scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).flatten()

    top_indices = similarity_scores.argsort()[-top_k:][::-1]

    print("\nTop", top_k, "Relevant Reviews:\n")

    for i in top_indices:

        print("Similarity Score:",
              round(similarity_scores[i], 4))

        print("Original Review:")
        print(df.iloc[i]["Text"])

        print("\nCleaned Review:")
        print(df.iloc[i]["Cleaned_Text"])

        if "ProductId" in df.columns:
            print("Product ID:", df.iloc[i]["ProductId"])

        if "Score" in df.columns:
            print("Score:", df.iloc[i]["Score"])

        if "Summary" in df.columns:
            print("Summary:", df.iloc[i]["Summary"])

        print("-" * 80)


retrieve_reviews("great product with fast shipping", 5)
retrieve_reviews("disappointed", 5)
```

4. Output

```
Number of reviews: 20
TF-IDF Matrix Shape: (20, 41)

Original Query: great product with fast shipping
Processed Query: great product fast shipping

Top 5 Relevant Reviews:

Similarity Score: 0.8538
Original Review:
This is a great product and it arrived with fast shipping.

Cleaned Review:
great product arrive fast shipping
--------------------------------------------------------------------------
------
Similarity Score: 0.5296
Original Review:
The shipping was fast and the product was exactly as described.

Cleaned Review:
shipping fast product exactly describe
--------------------------------------------------------------------------
------
Similarity Score: 0.5055
Original Review:
Great product, excellent quality and very fast delivery.
```

5. Learning outcome

- Learned how to use spaCy for Natural Language Processing.
- Learned to perform tokenization and lowercasing.
- Learned to remove stopwords and non-alphabetic tokens.
- Understood lemmatization and its importance in text preprocessing.
- Learned to represent reviews using TF-IDF vectors.
- Learned to calculate cosine similarity between queries and documents.
- Successfully retrieved the top-k relevant Amazon Fine Food Reviews.
- Learned how NLP preprocessing can improve the effectiveness of an Information Retrieval system.